

BigAnt Book

L'app oggi e la guida per i tester

Versione del 17 settembre 2026. Per soci e tester. Interfaccia scura, accenti arancioni, italiano e inglese.

BigAnt Book riunisce **prenotazioni, menu digitale e feedback** per ristoranti, bar e lidi. Ogni locale ha pagine cliente e pannello staff dedicati; i dati sono separati per locale nella stessa applicazione.

Il cliente prenota e legge il menu senza registrarsi. Il ristoratore gestisce agenda, sala, lista d'attesa, clienti e contenuti da un pannello web.

Stato della consegna: demo locale verificata. Il software si può provare sul proprio computer con due locali e dati inventati. Email, SMS e push restano simulati. L'attivazione con clienti reali richiede ancora lavoro e collaudo di produzione.

Repository e copia del progetto

[Apri BigAnt su GitHub](#)

URL da clonare: `https://github.com/lo1-afk23456/BigAnt.git`

GitHub distribuisce codice, migrazioni, seed e documentazione. Ogni tester ricrea il proprio database: le prove sul suo computer non compaiono su quello dei soci. Il repository non ospita un'app utilizzabile via link.

Il repository indicato è pubblico al momento di questa consegna: per clonarlo non serve un account. Se diventa privato, il proprietario deve concedere l'accesso al tester, che usa il proprio account e gestore credenziali. Nessun token va aggiunto al comando di clonazione.

Come leggere questa guida

Pagine 2-3: funzioni. Pagina 4: installazione Mac/Linux. Pagina 5: accessi e prima prova. Pagina 6: aggiornamenti e problemi. Pagina 7: produzione. Pagina 8: Windows e riferimenti.

Prenotazioni, sala e servizio

Il percorso del cliente

Una pagina unica rivela progressivamente persone, giorno, orario e contatti. Il numero di persone si sceglie con pulsanti. **Email e telefono sono entrambi obbligatori**; la richiesta speciale è facoltativa. Privacy e marketing sono separati e il marketing parte deselezionato.

Se oggi è chiuso o pieno, l'ingresso passa al primo giorno utile. Quando un giorno selezionato non ha posto, propone le due alternative disponibili più vicine nella finestra prenotabile, oppure invita a contattare il locale quando il recapito è configurato.

Il sistema controlla apertura, chiusure, anticipo, capienza, durata e ritmo degli arrivi. Se l'assegnazione automatica è attiva controlla anche un tavolo libero compatibile. La disponibilità è rivalidata in transazione alla conferma della prenotazione.

Santa Lucia conferma subito; il Lido riceve una richiesta da confermare. Il cliente ottiene una ricevuta a schermo e un link riservato per vedere/disdire entro il termine consentito.

Il lavoro dello staff

Agenda giornaliera con elenco e vista oraria, richieste da confermare e conteggi. Lo staff può inserire una telefonata, modificare prenotazione e note, assegnare tavoli e seguire gli stati: da confermare, confermata, al tavolo, completata, disdetta o assente. Il titolare regola orari, chiusure, capienza e parametri del servizio.

Tavoli e lista d'attesa

Tavoli fisici organizzati per zona e combinazioni consentite dal titolare. Una combinazione occupa tutti i componenti; l'assegnazione resta manuale e conserva la configurazione storica. Il percorso automatico cliente usa tavoli singoli.

La lista d'attesa si apre per giorno e servizio. Richiede cognome e coperti, mantiene l'ordine d'arrivo e mostra alternative compatibili. L'operatore decide chi accomodare; l'azione ricontrolla disponibilità e crea una prenotazione al tavolo. Completare libera i tavoli. Nessun SMS automatico alla fila.

Menu, feedback, clienti e notifiche

Menu digitale

Categorie e piatti in italiano/inglese, riordino, descrizioni, prezzi, allergeni, etichette dietetiche e piatti in evidenza. Foto JPEG, PNG o WebP fino a 5 MB, convertite in varianti WebP e private dei metadati.

Template scuro	Uso proposto
Essenziale	Trattoria, bistrot, menu sobrio
Pop	Pizzeria e locali informali
Elegante	Ristorante di ricerca
Pub	Birreria e burger bar

Il titolare sceglie template, colore principale e copertina. **L'occhio nasconde** il prodotto; **esaurito lo lascia visibile** in grigio. Il menu già aperto controlla le modifiche ogni 30 secondi quando è visibile, mantenendo il punto di lettura.

Feedback e card

Google e messaggio privato sono offerti con lo stesso peso prima di chiedere il voto. Il feedback privato è anonimo, con voto 1-5 e commento facoltativo; il pannello gestisce lettura e note interne. Le card hanno link e attivazione/disattivazione; la programmazione fisica NFC è esterna all'app.

I Place ID demo mostrano un esito locale: non pubblicano recensioni Google. Un clic verso Google non dimostra una recensione pubblicata. La stessa card ammette un invio ogni dieci minuti, condiviso fra gli ospiti.

Clienti e messaggi

Schede cliente con ricerca, note e storico; export CSV, anonimizzazione con storico conservato, audit e retention. L'informativa pubblica è una bozza da validare.

La coda persistente registra i messaggi prima dell'invio, gestisce promemoria, tentativi, limite SMS e fallback email. **Nella demo ogni invio è simulato**: il pannello non prova un recapito reale. La PWA è predisposta; installazione e push su telefoni fisici sono ancora da collaudare con HTTPS.

Installare e avviare su Mac o Linux

1. Preparare gli strumenti una sola volta

Servono Git, **Node 22.23.2** e **pnpm 10.32.1**. Su Mac, `git --version` può proporre l'installazione degli strumenti Xcode. Per Node usa la [pagina ufficiale dei download](#) scegliendo la versione richiesta, oppure [nvm](#). Se installi nvm, riapri il terminale prima del passo 2. Con nvm l'installazione globale di pnpm avviene nel profilo utente senza sudo.

Non servono Docker, PostgreSQL installato a mano o account email/SMS. Il primo avvio richiede Internet per dipendenze e binari; esegui i comandi come utente normale.

2. Clonare e installare

Apri Terminale e scegli una cartella dove conservare il progetto:

```
git clone https://github.com/lol-afk23456/BigAnt.git
cd BigAnt
```

Se usi nvm, dentro il progetto:

```
nvm install
nvm use
```

Poi, anche se hai installato Node dal download ufficiale:

```
node --version
npm install --global pnpm@10.32.1
pnpm --version
pnpm install --frozen-lockfile
pnpm local
```

`node --version` deve mostrare v22.23.2; `pnpm` deve mostrare 10.32.1. Attendi che il launcher segnali l'app pronta, quindi apri <http://localhost:3000>. Lascia il terminale aperto.

Il launcher crea l'ambiente locale e un segreto casuale, prepara PostgreSQL, migrazioni e seed, poi avvia web, API e worker. Il primo avvio può richiedere diversi minuti; quelli successivi riusano dati e build validi.

Porte e persistenza

Servono libere le porte 3000 e 3001; PostgreSQL usa 55432. Avvia una sola copia. Database e foto rimangono in `.local/`, esclusa da Git. Nel nuovo clone usa il terminale: il launcher a doppio clic del Mac originale dipende anche dal runtime locale non distribuito.

Entrare e fare la prima prova

Accessi demo

Locale	Email staff	Password
Trattoria Santa Lucia	owner@santalucia.test	bigant2026
Lido Miseno	owner@lidomiseno.test	bigant2026

Santa Lucia: 40 coperti, durata 90 minuti, ritmo 12, conferma automatica. Lido: 120 coperti, durata 120 minuti, ritmo 25, conferma manuale.

Dalla home scegli il locale. Ogni ingresso `/r/:slug` contiene prenotazione, menu e accesso staff; non sono due applicazioni separate. Per lo staff usa `/r/trattoria-santa-lucia/staff` oppure `/r/lido-miseno/staff` dopo `http://localhost:3000`.

Aggiungere esempi leggibili

Con `pnpm local` attivo, apri un secondo terminale nella cartella BigAnt. Se usi `nvm` esegui `nvm use`, poi:

```
pnpm demo:examples
```

Il comando stampa giorno/ora e servizio effettivi. Cerca **DEMO** nell'agenda: Giulia Rossi (2), Famiglia Bianchi (4), Gruppo Esposito (6). Nell'attesa: Costa (2), Conti (4), Gallo (6). Rispetta chiusure e posti, conserva le prove e non duplica la serie dello stesso giorno. Se salta un caso ne spiega il motivo. Al Lido i casi partono da confermare.

Percorso iniziale: circa 20-25 minuti

1. Apri un esempio DEMO nello staff e leggi le note della prova.
2. In incognito prenota Santa Lucia per dopodomani con dati inventati. Verifica conferma e presenza nell'agenda dello stesso giorno.
3. Usa il link di gestione e disdici: stato aggiornato e posto liberato.
4. Prenota il Lido, poi conferma dallo staff e assegna un tavolo compatibile.
5. Modifica un piatto: esaurito resta visibile; occhio lo nasconde entro il ciclo di aggiornamento. Ripristina il piatto.
6. Invia un feedback privato e segnale letto dal pannello.

Una finestra normale per lo staff e una in incognito per il cliente. Due staff contemporanei richiedono profili browser separati: le schede normali condividono la sessione.

Usa solo dati inventati; non contattare numeri demo. Il [protocollo completo](#) include risultati attesi e prove condizionate all'orario del servizio.

Riavvio, aggiornamenti e problemi comuni

Fermare e riprendere

Nel terminale del launcher premi **Ctrl+C**. Per riprendere, nella stessa cartella esegui `pnpm local`. Se hai riaperto il terminale e usi `nvm`, esegui prima `nvm use`. Dati e foto restano; un normale riavvio non azzerà le prove.

Ricevere gli aggiornamenti

Con app ferma e dalla cartella BigAnt:

```
git pull --ff-only
pnpm install --frozen-lockfile
pnpm local
```

Il launcher applica nuove migrazioni e ricompila quando necessario. Se Git segnala modifiche locali, conservale e chiedi assistenza: non usare un reset forzato. Ogni tester mantiene un database indipendente; Git non sincronizza le sue prenotazioni con altri computer.

Se qualcosa non parte

Sintomo	Controllo / azione
Clone negato	URL e accesso al repository. Se privato, serve l'invito e il proprio accesso GitHub.
Strumenti assenti o versione errata	Riapri il terminale; verifica Node 22.23.2 e pnpm 10.32.1. Con <code>nvm</code> esegui <code>nvm install</code> e <code>nvm use</code> .
Porta occupata	Ferma l'altra copia o identifica il processo prima di chiuderlo. Non avviare un secondo launcher.
Nessuna email ricevuta	La modalità locale simula gli invii: controlla Notifiche, senza attivare provider reali.

Per ripartire volutamente da zero esiste `pnpm demo:reset --confirm`, ad app ferme e con DB locale attivo. **Cancella tutte le prove dei due demo**: non fa parte dell'avvio normale. Non cancellare `.local/` per risolvere un errore.

Segnalare un difetto utile

Invia versione (`git rev-parse --short HEAD`), sistema/browser, locale, giorno/servizio, passi, atteso e osservato. Aggiungi screenshot con soli dati inventati. Non condividere `.env`, token di gestione, CSV con persone reali o l'intera cartella `.local/`.

Cosa manca prima dell'uso reale

Evidenze disponibili

Ultimo cancello applicativo locale del 17 settembre: typecheck, lint, test e build verdi; **104 test backend**, inclusi 26 di isolamento tenant, e **7 scenari browser**. Queste evidenze descrivono il codice verificato sul Mac; non certificano la produzione o il funzionamento su tutti i sistemi dei tester.

Collegare servizi esterni non basta

Area	Attività ancora necessaria
Ambiente EU	Hosting, PostgreSQL, worker, immagini e log persistenti; deploy, riavvio e rollback.
Accessi e rete	Credenziali uniche e provisioning/recupero; HTTPS, CSP/HSTS e rate limit corretti dietro proxy.
Continuità	Backup di DB e immagini, ripristino realmente provato, salute DB/coda e alert ricevuti.
Messaggi	Dominio mittente, DNS e account verificati EU; ricezione email/SMS, quota/fallback e gestione esiti incerti.
Telefoni e privacy	Installazione/push Android e iPhone, test e ruoli privacy validati, contratti e retention.
Pilot	Cancello sul rilascio, carico realistico e servizio con un ristoratore esterno ai soci.

Account necessari: dominio/DNS, hosting e database EU, storage persistente, backup, email transazionali, caselle assistenza e monitoraggio. SMS e push richiedono verifiche proprie. La coda usa già PostgreSQL: un servizio aggiuntivo di code non è obbligatorio.

La [guida alla produzione](#) elenca P01-P12 e le evidenze per chiuderli. Il [report servizi esterni](#) separa fornitori candidati da configurazioni approvate.

Funzioni che non ci sono oggi

Console agenzia per gestire tutti i clienti, piantina della sala a blocchi Pro, pagamenti/acconti, ordinazioni, cassa/POS, sincronizzazione TheFork, fidelity, multi-sede, app nativa e assistente vocale/AI. Non sono necessarie per la prova locale di questa consegna.

GitHub permette ai soci di provare il progetto in locale. Per una prova comune via link occorre un ambiente remoto di collaudo separato, con HTTPS e dati inventati; non basta lasciare acceso il Mac di un socio.

Aggiornamento GitHub - 17 settembre

Tipi, lint, test backend e build verdi. Benchmark menu: 95/100, LCP 2,35 s; limite <2 s ancora aperto. Dettagli in PROGRESS.

Windows e riferimenti operativi

Windows: percorso WSL2, ancora da collaudare

L'esecuzione nativa Windows del launcher non è verificata. Per un tester Windows consigliamo Ubuntu in WSL2; anche questo percorso deve avere una prima prova sul dispositivo reale. Il Mac è l'ambiente già provato; il workflow Linux è predisposto.

Su Windows compatibile, apri PowerShell come amministratore e segui la [guida Microsoft WSL](#):

```
wsl --install
```

Riavvia quando richiesto. Apri Ubuntu e crea l'utente Linux. Nel terminale Ubuntu:

```
sudo apt update  
sudo apt install -y git curl  
cd ~
```

Installa nvm seguendo la sua guida ufficiale e riapri Ubuntu. Poi segui i comandi della pagina 4: clone, nvm install/use, pnpm e pnpm local. Conserva il progetto nella home Linux, non nella cartella di sistema Windows. Esegui il launcher come utente normale, senza sudo. Apri localhost:3000 dal browser; se non è raggiungibile registra il problema WSL prima di modificare la rete.

Link utili e fonti

- [Repository BigAnt](#) - codice e documentazione.
- [Indice dei documenti](#) - stato, scelte e specifica.
- [Dati demo](#) - cosa cercare e come aggiornarlo.
- [Protocollo di test](#) - T01-T20 ed esiti attesi.
- [Clonare un repository, GitHub](#) - accesso e clonazione.
- [Node.js](#) e [nvm](#) - installazione del runtime.
- [Installazione pnpm 10](#) - usa la versione 10.32.1 del progetto.
- [embedded-postgres](#) - PostgreSQL incluso e binari per piattaforma.
- [Microsoft WSL](#) - installazione Windows/Ubuntu.

Le versioni dei comandi sono fissate dal progetto, non dalla versione più recente dei singoli strumenti. L'elenco delle funzioni deriva dal codice e dallo stato verificato in PROGRESS, non da promesse sui moduli futuri.